

Фестиваль

Найра на фестивалі грає в гру, головним призом якої є поїздка до Лагуни Колорадо. Гра полягає у використанні жетонів для купівлі купонів. Купівля купона може призвести до отримання додаткових жетонів. Мета — отримати якомога більше купонів.

Вона починає гру з A жетонами. Є N купонів, пронумерованих від 0 до $N - 1$. Найрі потрібно заплатити $P[i]$ жетонів ($0 \leq i < N$), щоб купити купон i (і вона повинна мати принаймні $P[i]$ жетонів перед покупкою). Вона може купити кожен купон не більше одного разу.

Крім того, кожному купону i ($0 \leq i < N$) присвоюється **тип**, який позначається $T[i]$, який є цілим числом **від 1 до 4 включно**. Після того, як Найра купила купон i , кількість жетонів, що залишилась у неї, множиться на $T[i]$. Формально, якщо у неї є X жетонів у певний момент гри, і вона купує купон i (який коштує $X \geq P[i]$), то після покупки у неї буде $(X - P[i]) \cdot T[i]$ жетонів.

Ваше завдання - визначити, які купони повинна купити Найра і в якому порядку, щоб максимізувати загальну кількість **купонів**, які вона отримає в кінці. Якщо є кілька послідовностей покупок, які досягають цієї мети, ви можете вивести про будь-яку з них.

Деталі реалізації

Вам потрібно реалізувати наступну функцію.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
                             std::vector<int> T)
```

- A : початкова кількість жетонів, яку має Найра.
- P : масив довжиною N , що містить ціни купонів.
- T : масив довжиною N , що містить типи купонів.
- Ця функція викликається рівно один раз для кожного тесту.

Функція повинна повертати масив R , який задає покупки Найри наступним чином:

- Довжина масиву R повинна дорівнювати максимальній кількості купонів, які вона може купити.
- Елементами масиву є індекси купонів, які вона повинна купити, у хронологічному порядку. Тобто, спочатку вона купує купон $R[0]$, потім купон $R[1]$ і так далі.
- Всі елементи масиву R повинні бути унікальними.

Якщо жоден купон не може бути куплений, то R повинен бути порожнім масивом.

Обмеження

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ для усіх i , таких, що $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ для усіх i таких, що $0 \leq i < N$.

Підзадачі

Підзадача	Бали	Додаткові обмеження
1	5	$T[i] = 1$ для усіх i , таких, що $0 \leq i < N$.
2	7	$N \leq 3000$; $T[i] \leq 2$ для усіх i , таких, що $0 \leq i < N$.
3	12	$T[i] \leq 2$ для усіх i , таких, що $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Найра може купити всі N купонів (у певному порядку).
6	16	$(A - P[i]) \cdot T[i] < A$ для усіх i , таких, що $0 \leq i < N$.
7	18	Без додаткових обмежень

Приклади

Приклад 1

Розглянемо наступний виклик.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Спочатку у Найри є $A = 13$ жетонів. Вона може купити 3 купони у порядку, показаному нижче:

Куплений купон	Ціна купона	Тип купона	Кількість жетонів після покупки
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

У цьому прикладі Найра не може купити більше, ніж 3 купони, і описана вище послідовність покупок - єдиний спосіб, у який вона може купити 3 купони.

Отже, функція повинна повернути $[2, 3, 0]$.

Приклад 2

Розглянемо наступний виклик.

```
max_coupons(9, [6, 5], [2, 3])
```

У цьому прикладі Найра може купити обидва купони у будь-якому порядку. Отже, функція повинна повернути або $[0, 1]$, або $[1, 0]$.

Приклад 3

Розглянемо наступний виклик.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

У цьому прикладі Найра має один жетон, якого недостатньо для купівлі купонів. Отже, функція повинна повернути $[]$ (порожній масив).

Приклад градера

Формат вхідних даних:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Формат вихідних даних:

```
S
R[0] R[1] ... R[S-1]
```

Тут S - довжина масиву R , що повертається `max_coupons`.